

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)

Институт информационных технологий, математики и механики

Направление подготовки: «Программная инженерия»

ОТЧЁТ
по производственной (преддипломной) практике

**ПАРАЛЛЕЛЬНАЯ РЕАЛИЗАЦИЯ МЕТОДА
QR-РАЗЛОЖЕНИЯ**

Выполнила: студентка группы
3824М1Пр1

А.Д. Коробейникова
Подпись

Проверил: д.т.н., доц.

К.А. Баркалов
Подпись

Нижний Новгород

2026 г.

Содержание

Введение	3
Постановка задачи	5
Описание алгоритмов	6
Метод Гивенса	6
Базовая реализация	8
Хранение матриц Q и R в одной структуре	10
Использование SIMD	11
QR-разложение Хессенберговой матрицы	13
Описание программы	15
Метод Гивенса	17
Базовая реализация	17
Хранение Q и R в одной структуре	23
Использование SIMD	27
QR-разложение Хессенберговой матрицы	35
Заключение	39
Список литературы	40
Приложение 1.	41
Приложение 2.	43
Приложение 3.	45

Введение

Зачастую научно-технические расчёты требуют решения задач линейной алгебры. Моделирование многих физических процессов сводится к системам дифференциальных уравнений, которые в численном решении приводятся к виду систем линейных алгебраических уравнений. Поэтому проблема быстрой обработки таких больших объёмов данных очень актуальна. В этой работе рассматривается алгоритм QR-разложения для представления матрицы в более удобном для быстрых расчётов виде. Кроме того, QR-разложение также является основой одного из методов поиска собственных векторов и чисел матрицы — QR-алгоритма [1]. С помощью этого разложения можно также решить линейную задачу наименьших квадратов и систему линейных алгебраических уравнений.

QR-разложение матрицы – представление матрицы A размера $p \times m$, где $p \geq m$ в виде произведения унитарной (или ортогональной в вещественном виде) Q ($Q^T Q = E$) и верхнетреугольной матрицы R . При $p=m$ матрица Q унитарная. Если при этом A невырождена, то QR-разложение единственно и матрица R может быть выбрана так, чтобы её диагональные элементы были положительными вещественными числами. В частном случае, когда матрица A состоит только из вещественных чисел, матрицы Q и R также могут быть выбраны вещественными, причём Q является ортогональной [1]. На сегодняшний день применяется несколько методов для вычисления QR-разложения матриц [2]. Для ряда методов существуют эффективные программные реализации [3 - 5].

QR-разложение может быть получено различными методами. Проще всего оно может быть вычислено в процессе ортогонализации Грама-Шмидта [6]. Другие методы QR-разложения основаны на отражениях Хаусхолдера и вращениях Гивенса [2, 7]. Все перечисленные методы имеют ряд ограничений для применения [8]:

- Процесс ортогонализации Грама-Шмидта имеет сложность $O(n^3)$; практически не поддаётся распараллеливанию; требует явного хранения Q (n^2 элементов); численно неустойчив, в большинстве случаев приводит к нарушению ортогональности. Модифицированный алгоритм чуть лучше, но и он имеет этот недостаток.
- Отражения Хаусхолдера имеют сложность приблизительно $\frac{4}{3}n^3$; для достижения наибольшей эффективности требуется более сложная блочная реализация; вектора отражений можно хранить под диагональю матрицы R , экономя память; для построения явной Q все равно требуется $O(n^2)$ памяти и дополнительных вычислений; численно устойчив. Является основным методом для разложения плотных матриц

общего вида. Из-за действий над целыми столбцами неэффективен для разреженных матриц, так как не учитывает уже занулённые значения.

- Вращения Гивенса имеют сложность $2n^3$; каждое вращение затрагивает только две строки, что позволяет эффективно распараллеливать разные вращения на одном столбце; вектора вращений можно хранить очень компактно или даже не хранить явно; численно устойчив; лучше всего подходит для разреженных и ленточных матриц, так как позволяет обнулять отдельные элементы, не затрагивая другие; в связи с этим медленнее работает для плотных матриц. Этот метод также используется для обновления уже готового разложения при добавлении новой строки или столбца.

В этой работе приводятся параллельные реализации QR-разложения по методам Хаусхолдера и Гивенса с применением технологий параллелизма OpenMP, а также решение задачи QR-разложения Хессенберговой матрицы.

Постановка задачи

Необходимо оптимизировать полученные на предыдущем этапе алгоритмы QR-разложения произвольной матрицы A по методу вращений Гивенса и отражений Хаусхолдера. Далее реализовать решение задачи QR-разложения Хессенберговой матрицы. Сравнить результаты с существующим решением.

Для разработки использовать язык Си++ и технологии OpenMP для ускорения.

Сгенерировать целочисленную матрицу A размером $n \times n$. Протестировать работу алгоритма на разных размерностях матрицы n : от небольших размеров (100 элементов), до больших объёмов данных ($n \gg 1000$). Протестировать работу на различном количестве потоков. Сравнить работу алгоритма с известными реализациями в сторонних библиотеках.

Посчитать абсолютную $\|QR - A\|$ и относительную $\frac{\|QR - A\|}{\|A\|}$ ошибки для разложения. Критерием правильности считать близость ошибок к какому-либо малому ϵ , например, $1 \cdot 10^{-7}$ – такую точность знаков после запятой гарантирует тип float в Си++.

Описание алгоритмов

Метод Гивенса

Метод Гивенса также выполняет разложение матрицы A на Q и R , но с помощью операции вращения. Каждая из матриц вращения Гивенса представлена позициями столбцов j и i и параметрами $\cos$ и $\sin$ вращения:

$$G_{ji} = \begin{bmatrix} 1 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \dots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & 0 \\ 0 & \dots & 1 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & c & 0 & \dots & 0 & -s & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & s & 0 & \dots & 0 & c & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Подматрица $\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$ расположена на строках и столбцах с номерами j и i , является ортогональной ($G^T G = E$).

Если дан вектор $a = [a_1 \dots a_n]^T \in \mathbb{R}^n, s = \sqrt{a_j^2 + a_i^2} \neq 0$, то выбрав $\cos\theta = \frac{a_j}{\sqrt{a_j^2 + a_i^2}}, \sin\theta = \frac{-a_i}{\sqrt{a_j^2 + a_i^2}}$ можно обнулить i компоненту вектора a :

$$\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} a_j \\ a_i \end{bmatrix} = \begin{bmatrix} \cos\theta \cdot a_j - \sin\theta \cdot a_i \\ \sin\theta \cdot a_j + \cos\theta \cdot a_i \end{bmatrix} = \begin{bmatrix} \frac{a_j^2 + a_i^2}{\sqrt{a_j^2 + a_i^2}} \\ \frac{-a_j \cdot a_i + a_j \cdot a_i}{\sqrt{a_j^2 + a_i^2}} \end{bmatrix} = \begin{bmatrix} \sqrt{a_j^2 + a_i^2} \\ 0 \end{bmatrix}.$$

На каждом шаге алгоритма две строки j и i преобразуемой матрицы подвергаются преобразованию вращения. При этом параметр такого преобразования подбирается так, чтобы один из поддиагональных элементов преобразуемой матрицы стал нулевым. Сначала в нуль последовательно обращаются элементы 1-го столбца, затем 2-го, и т.д. до $n-1$ -го, после чего получившаяся матрица — это и есть матрица R . Сам шаг алгоритма распадается на две части: подбор параметра вращения и выполнение вращения над двумя строками матрицы. Поскольку слева от "рабочего" столбца элементы вращаемых строк равны 0, то там выполнять вращение не нужно.

Вращение элементов строк в текущем столбце выполняется одновременно с подбором параметра вращения. Таким образом, вторая часть шага заключается в выполнении вращения двумерных векторов, составленных из элементов вращаемых строк в столбцах справа от "рабочего".

Как уже было сказано выше, для получения верхнетреугольной матрицы R последовательно занулим все элементы ниже главной диагонали, то есть последовательно домножим матрицу A слева на матрицы вращения:

$$R = G_{n-1,n} G_{n-2,n} \dots G_{2n} G_{1n} \dots G_{24} G_{23} \dots G_{13} G_{12} A.$$

Для получения Q:

- Ясно, что изначально $R = A$, $Q = E$, в виду чего, очевидно, выполняется $A = QR$.
- Теперь выполняем одно вращение, домножая R слева на матрицу G_{12} , получим $QG_{12}R$. Чтобы равенство $A = QR$ не нарушалось, домножим Q справа на G_{12}^{-1} , что из-за ортогональности матрицы вращения означает, что её обратная матрица может быть заменена на транспонированную, G_{12}^T . Получим $EG_{12}^T G_{12} R = A$, где $G_{12}^T G_{12} = E$.
- На каждом шаге алгоритма будем домножать полученную с предыдущего шага матрицу A слева на новую матрицу вращения, тем самым накапливая R.
- Тогда матрицу Q получим как результат последовательного перемножения друг на друга справа транспонированных матриц вращения:

$$\begin{aligned} Q &= (G_{n-1,n} G_{n-2,n} \dots G_{2n} G_{1n} \dots G_{24} G_{23} \dots G_{13} G_{12} A)^T \\ &= G_{12}^T G_{13}^T \dots G_{1n}^T G_{23}^T G_{24}^T \dots G_{2n}^T \dots G_{n-2,n}^T G_{n-1,n}^T \end{aligned}$$

- В результате получим верхнетреугольную матрицу R и унитарную матрицу Q.

При этом домножение на матрицу вращения (j,i) слева будет вращать строки j и i, а справа — столбцы j и i. Таким образом, перемножение матриц затрагивает не все NxN элементов, а только 2(N-j), где j — номер текущего зануляемого столбца, элементов при вращении строк для вычисления матрицы R (т.к. при вращении элементы слева от текущего столбца обнуляются) и 2N элементов при вращении столбцов для накопления Q.

Базовая реализация

Таким образом, базовая реализация алгоритма разложения двумерной матрицы A в двумерные верхнетреугольную R и ортогональную Q будет следующим:

1. для всех диагональных элементов (i, j) по очереди слева направо сверху вниз вычисляется параметр вращения:

$$\cos\theta = c = \frac{a_{jj}}{\sqrt{a_{jj}^2 + a_{ij}^2}}, \sin\theta = s = \frac{-a_{ij}}{\sqrt{a_{jj}^2 + a_{ij}^2}}$$

2. матрица A домножается слева на матрицу вращения G_{ji} :

$$G_{ji} \dots G_{12} A =$$

$$\begin{bmatrix} 1 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \dots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & c & 0 & \dots & 0 & -s & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & s & 0 & \dots & 0 & c & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 1 \end{bmatrix} \cdot \begin{bmatrix} a_{11} & \dots & a_{1,j-1} & a_{1j} & a_{1,j+1} & \dots & a_{1,i-1} & a_{1i} & a_{1,i+1} & \dots & a_{1n} \\ \dots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & a_{j-1,j-1} & a_{j-1,j} & a_{j-1,j+1} & \dots & a_{j-1,i-1} & a_{j-1,i} & a_{j-1,i+1} & \dots & a_{j-1,n} \\ 0 & \dots & 0 & a_{jj} & a_{j,j+1} & \dots & a_{j,i-1} & a_{ji} & a_{j,i+1} & \dots & a_{jn} \\ 0 & \dots & 0 & 0 & a_{j+1,j+1} & \dots & a_{j+1,i-1} & a_{j+1,i} & a_{j+1,i+1} & \dots & a_{j+1,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & a_{i-1,j+1} & \dots & a_{i-1,i-1} & a_{i-1,i} & a_{i-1,i+1} & \dots & a_{i-1,n} \\ 0 & \dots & 0 & a_{ij} & a_{i,j+1} & \dots & a_{i,i-1} & a_{ii} & a_{i,i+1} & \dots & a_{in} \\ 0 & \dots & 0 & a_{i+1,j} & a_{i+1,j+1} & \dots & a_{i+1,i-1} & a_{i+1,i} & a_{i+1,i+1} & \dots & a_{i+1,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & a_{n,j} & a_{n,j+1} & \dots & a_{n,i-1} & a_{ni} & a_{n,i+1} & \dots & a_{nn} \end{bmatrix}$$

Красным отмечена главная диагональ, ниже которой все элементы должны быть занулены.

Зелёным отмечен текущий зануляемый элемент a_{ij} , все столбцы левее его уже занулены,

также занулены все элементы того же столбца выше a_{ij} . Синим выделены $\cos\theta$ (G_{jj} и G_{ii}) и $\sin\theta$ (G_{ji} и G_{ij}) и строки, которые будут вращаться.

Для вычисления новых значений в строках A_j и A_i перемножаются строка G_j и столбцы A_k , $k \in [j, \dots, N]$ и G_i и столбцы A_k , $k \in [j, \dots, N]$, то есть при этом в строках A_j и A_i ненулевые значения будут только правее, начиная с j столбца, что уменьшает количество вычислений.

Вычисление нового значения заключается в сложении двух умножений:

$$a_{ik} = a_{jk} \cdot c - a_{ik} \cdot s,$$

$$a_{jk} = a_{jk} \cdot s + a_{ik} \cdot c.$$

3. матрица Q вычисляется с помощью полученной матрицы вращения домножением на неё справа:

$$Q = G_{12}^T \dots G_{ji}^T$$

$$\begin{bmatrix} Q_{11} & \dots & Q_{1,j-1} & Q_{1j} & Q_{1,j+1} & \dots & Q_{1,i-1} & Q_{1i} & Q_{1,i+1} & \dots & Q_{1n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{j-1,1} & \dots & Q_{j-1,j-1} & Q_{j-1,j} & Q_{j-1,j+1} & \dots & Q_{j-1,i-1} & Q_{j-1,i} & Q_{j-1,i+1} & \dots & Q_{j-1,n} \\ Q_{j,1} & \dots & Q_{j,j-1} & Q_{jj} & Q_{j,j+1} & \dots & Q_{j,i-1} & Q_{ji} & Q_{j,i+1} & \dots & Q_{jn} \\ Q_{j+1,1} & \dots & Q_{j+1,j-1} & Q_{j+1,j} & Q_{j+1,j+1} & \dots & Q_{j+1,i-1} & Q_{j+1,i} & Q_{j+1,i+1} & \dots & Q_{j+1,n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{i-1,1} & \dots & Q_{i-1,j-1} & Q_{i-1,j} & Q_{i-1,j+1} & \dots & Q_{i-1,i-1} & Q_{i-1,i} & Q_{i-1,i+1} & \dots & Q_{i-1,n} \\ Q_{i,1} & \dots & Q_{i,j-1} & Q_{ij} & Q_{i,j+1} & \dots & Q_{i,i-1} & Q_{ii} & Q_{i,i+1} & \dots & Q_{in} \\ Q_{i+1,1} & \dots & Q_{i+1,j-1} & Q_{i+1,j} & Q_{i+1,j+1} & \dots & Q_{i+1,i-1} & Q_{i+1,i} & Q_{i+1,i+1} & \dots & Q_{i+1,n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{n,1} & \dots & Q_{n,j-1} & Q_{n,j} & Q_{n,j+1} & \dots & Q_{n,i-1} & Q_{ni} & Q_{n,i+1} & \dots & Q_{nn} \end{bmatrix} \cdot \begin{bmatrix} 1 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & c & 0 & \dots & 0 & -s & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & s & 0 & \dots & 0 & c & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Красным отмечена главная диагональ. Синим выделены $\cos\theta$ (G_{jj} и G_{ii}) и $\sin\theta$ (G_{ji} и G_{ij}) и столбцы, которые будут вращаться.

Для вычисления новых значений в столбцах Q_j и Q_i перемножаются строка Q_k , $k \in [1, \dots, N]$ и столбцы G_j , и Q_k , $k \in [1, \dots, N]$ и столбцы G_i , количество вычислений здесь не уменьшается. Вычисление нового значения заключается в сложении двух умножений:

$$q_{ki} = q_{kj} \cdot s + q_{ki} \cdot c,$$

$$q_{kj} = q_{kj} \cdot c - q_{ki} \cdot s.$$

Хранение матриц Q и R в одной структуре

Если использовать подход, при котором матрица Q вычисляется там же, где и матрица R, во время основного цикла вращения, возможно использование специализированной структуры данных с быстрым доступ к элементам обеих матриц. Эта идея берёт за основу тот факт, что алгоритм обращается к i и j строкам матрицы R и одновременно к i и j столбцам матрицы Q, и если хранить матрицу Q транспонированной, произойдёт обращение к одним и тем же элементам R_{ik} и Q^T_{ik} , R_{jk} и Q^T_{jk} во время вращения строк (транспонированных столбцов, в случае матрицы Q).

Небольшой выигрыш по ускорению может дать совместное попарное хранение элементов Q и R благодаря оптимизации доступа к памяти. При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи.

Использование SIMD

При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи.

Для максимального эффекта по части улучшения обращения к памяти можно использовать векторизацию и SIMD.

Векторизация — вид распараллеливания программного кода, при котором однопоточные программы, исполняющие одну операцию над одним элементом (single instruction-single data) в каждый момент времени, модифицируются таким образом, чтобы они могли выполнять более одной операции одновременно. Скалярные операции, обрабатывающие пару аргументов заменяются на векторные аналоги, способные производить одну и ту же операцию над массивом данных в единицу времени. Векторная обработка данных широко применяется как в лабораторных суперкомпьютерах, так и в обычных домашних компьютерах.

SIMD (Single Instruction, Multiple Data) — это технология из области компьютерных высокоэффективных вычислений, позволяющая с помощью одной машинной инструкции (single instruction) процессора обрабатывать несколько элементов данных (multiple data). SIMD-инструкции используют специальные векторные регистры, в которые помещается несколько элементов. Чтение/запись, арифметические операции над этими регистрами выполняются процессором одновременно. Количество данных, которые могут храниться и обрабатываться одновременно, зависит от размеров регистра и данных. Некоторые распространённые SIMD-расширения современных процессоров:

- SSE (Streaming SIMD Extensions) — 128-битные регистры, позволяющие обрабатывать четыре 32-битных значения с плавающей запятой одинарной точности в один момент времени. Расширение SSE2 позволило использовать два 64-битных значения с плавающей точкой двойной точности, или два 64-битных целых числа, или четыре 32-битных, или восемь 16-битных коротких целых чисел, или шестнадцать 8-битных байтов или символов.
- AVX (Advanced Vector Extensions) — 256-битные регистры, поддерживающие более сложные операции. В архитектуре x86-64 AVX поддерживает шестнадцать 256-битных регистров YMM. Каждый регистр YMM может хранить и выполнять одновременные операции над:

- восемь 32-битными числами с плавающей запятой одинарной точности;
- четыре 64-битными числами с плавающей запятой двойной точности.
- AVX-512 — расширение с увеличенным размером векторных регистров, которое увеличивает размер каждого регистра до 512 бит, что, очевидно, позволяет обрабатывать больше данных за одну операцию.

Автоматическую векторизацию некоторым образом обеспечивают компиляторы.

Современные представители, GCC, Clang, MSVC, могут анализировать код и заменять скалярные инструкции на векторные с использованием SIMD-инструкций.

Также компилятору с помощью директив можно указать принудительное использование simd-инструкций. `#pragma simd` для компиляторов Intel C/C++ позволяет указать, что компилятор должен явно векторизовать цикл, а `#pragma omp simd` — директива для той же цели из стандарта OpenMP.

Кроме того, можно использовать так называемые `intrinsic`'и, встроенные функции для работы с SIMD-инструкциями. Они позволяют использовать команды расширений с синтаксисом, похожим на вызов C-подобных функций, при этом компилятор сам выполнит оптимизацию кода.

Для векторизации кода и использования `simd` матрицы Q и R будем хранить в ленточном виде, записав в одномерный массив матрицы по строкам (матрица Q снова будет храниться транспонированной).

Ожидается, что SIMD-версия даст значительный прирост к производительности алгоритма, уменьшив время исполнения программы, так как она обеспечивает лучшее использование аппаратных возможностей современных процессоров.

QR-разложение Хессенберговой матрицы

Матрица Хессенберга — вид квадратной матрицы, обобщающий треугольные матрицы. Названа так в честь немецкого математика, Карла Хессенберга.

Верхняя матрица Хессенберга — это квадратная матрица

$$H = \begin{pmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2n} \\ 0 & h_{32} & h_{33} & \cdots & h_{3n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{nn-1} & h_{nn} \end{pmatrix}$$

Аналогично определяется нижняя матрица Хессенберга, как квадратная матрица, при транспонировании которой получается верхняя матрица Хессенберга:

$$H = \begin{pmatrix} h_{11} & h_{12} & 0 & \cdots & 0 \\ h_{21} & h_{22} & h_{23} & \cdots & \vdots \\ h_{31} & h_{32} & h_{33} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & h_{n-1n} \\ h_{n1} & h_{n2} & h_{n3} & \cdots & h_{nn} \end{pmatrix}$$

Матрица, являющаяся одновременно и верхней, и нижней матрицами Хессенберга, трёхдиагональна.

$$A = \begin{pmatrix} a_{11} & a_{12} & 0 & \cdots & \cdots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \cdots & \vdots \\ 0 & a_{32} & a_{33} & a_{34} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots \\ 0 & \cdots & \cdots & 0 & a_{nn-1} & a_{nn} \end{pmatrix}$$

Матрицы Хессенберга получаются как продукт методов подпространства Крылова в процессе построения ортогональных базисов, а также в задаче на нахождение собственных значений матрицы QR-методом.

С помощью алгоритма QR-разложения на основе вращений Гивенса решение задачи разложения Хессенберговой матрицы выглядит следующим образом:

Пусть верхняя хессенбергова матрица A имеет размер 6x6, после первых двух шагов:

$$G(2,3, \theta_2)^T \cdot G(1,2, \theta_1)^T \cdot A = \begin{pmatrix} x & x & x & x & x & x \\ 0 & x & x & x & x & x \\ 0 & 0 & x & x & x & x \\ 0 & 0 & x & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix}$$

Следующий шаг должен обнулить элемент с индексами 4 и 3 с помощью вращения $G(3, 4, \theta_3)$:

$$G(3,4,\theta_3)^T \cdot G(2,3,\theta_2)^T \cdot G(1,2,\theta_1)^T \cdot A = \begin{pmatrix} x & x & x & x & x & x \\ 0 & x & x & x & x & x \\ 0 & 0 & x & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix}$$

Для верхней хессенберговой матрицы алгоритм QR-разложения будет следующим:

$$Q^T \cdot A = R,$$

$$Q = G_1 \cdot \dots \cdot G_{n-1}, \text{ где } G_j = G(j, j+1, \theta_j).$$

Описание программы

Для матриц оптимизированные версии алгоритмов используют следующие типы:

- стандартный вектор длины $N \times N$, матрица записана по строкам;
- TransposedMatrix, транспонированная матрица, записанная по столбцам. В основе такой же стандартный вектор длины $N \times N$;
- QRMatrix, матрица, хранящая в себе элементы структуры для одновременного доступа к одному элементу Q и R в одной и той же позиции для попадания в кэш.

Для удобства тестирования, методы Хаусхолдера и Гивенса представлены методами классов, имплементирующих интерфейс IQRSolver с одним абстрактным методом для QR разложения:

```
virtual void QR_decomposition(const std::vector<std::vector<T>>& A, std::vector<std::vector<T>>& Q,
std::vector<std::vector<T>>& R) = 0;
```

В предыдущем семестре была выявлена проблема быстрого накопления ошибки для типа данных, представляющих в C++ числа с плавающей точкой одинарной точности, float. Изменение на double, числа с плавающей точкой двойной точности, дало стабильность для порядка абсолютной ошибки — не менее 10 знаков после запятой вне зависимости от размеров данных. При этом оптимальные версии работают без серьёзного замедления даже на данных более 1000×1000 элементов. Шаблонный код позволяет легко управлять этим параметром через механизм создания псевдонимов для типов данных:

```
typedef double currentType;
```

Другой параметр, количество потоков, по умолчанию равный максимальному количеству логических ядер в системе, на которой я тестирую:

```
size_t thread_num = 8;
```

Функция `main` запускает вычисления для нескольких тест-кейсов с методом разложения, описанием метода и размером данных (последним параметром в тестирующую функцию передаётся флаг для печати входных и выходных данных в файлы для сверения с SciLab реализацией):

```
auto methods = std::map<int, testParams>{};

methods[0] = {
    new HouseholderMethodBasic<currentType>(),
    "Householder basic version",
    { 100, 200, 300, 400, 500 },
};

methods[1] = {
    new HouseholderMethodWithoutMatrixMults<currentType>(),
    "Householder without matrix multiplications",
    { 100, 200, 300, 400, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000 },
};

methods[2] = {
    new HouseholderMethodWithNormW < currentType>,
    "Householder with normal w in-place",
    { 100, 200, 300, 400, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000 },
};

methods[3] = {
    new GivensMethodBasic<currentType>(),
    "Givens basic version",
    { 100, 200, 300, 400, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000 },
};

for (const auto& method : methods)
{
    std::cout << method.second.description << std::endl;
    for (auto&& size : method.second.sizes)
    {
        QR_decomposition_test_with_generated_matrix(method.second.solver, size, true);
    }
}
```


Метод Гивенса

Базовая реализация

Базовая версия выполнена по следующему алгоритму:

1. Занулить $(N / 2 - N)$ элементов, для этого в цикле по столбцам $j, j \in [0, N - 1)$ для всех строк $i, i \in [j + 10, N)$ выполнить вращения элемента (i, j) :
 1. Проверить, нужно ли занулять элемент (i, j) . Если он уже близок к нулю, пропустить это вращение.
 2. Вычислить c и s по формулам:

$$\cos\theta = c = \frac{a_{jj}}{\sqrt{a_{jj}^2 + a_{ij}^2}}, \sin\theta = s = \frac{-a_{ij}}{\sqrt{a_{jj}^2 + a_{ij}^2}}.$$

3. Вращать две строки i и j (элементы a_{jk} и $a_{ik}, k \in [i, N)$), выполнив вычисления по формулам:

$$a_{ik} = a_{jk} \cdot c - a_{ik} \cdot s,$$

$$a_{jk} = a_{jk} \cdot s + a_{ik} \cdot c.$$

Это можно сделать параллельно для всех k .

4. С помощью матрицы вращения накапливать матрицу Q . Для этого вращать два столбца i и j (элементы q_{kj} и $q_{ki}, k \in [0, N)$), выполнив вычисления по формулам:

$$q_{ki} = q_{kj} \cdot s + q_{ki} \cdot c,$$

$$q_{kj} = q_{kj} \cdot c - q_{ki} \cdot s.$$

К вычислению Q может быть два подхода.

Можно вычислять вращения столбцов для Q в том же месте, где вычисляется R. В результате код будет выглядеть следующим образом:

```
for (int j = 0; j < N - 1; ++j) {
    for (int i = j + 1; i < N; ++i) {
        // приведение к double для точных проверок и вычислений
        double Rjj = static_cast<double>(R[j][j]);
        double Rij = static_cast<double>(R[i][j]);

        // проверки малости (пороги для double)
        if (std::abs(Rij) < 1e-11)
            continue;

        double sqrt_val = std::sqrt(Rjj * Rjj + Rij * Rij);
        if (std::abs(sqrt_val) < 1e-11)
            continue;

        double c = Rjj / sqrt_val;
        double s = -Rij / sqrt_val;

#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int k = 0; k < N; ++k) {
            T temp = static_cast<T>(R[j][k] * c - R[i][k] * s);
            R[i][k] = static_cast<T>(R[j][k] * s + R[i][k] * c);
            R[j][k] = temp;

            temp = static_cast<T>(c * Q[k][j] - s * Q[k][i]);
            Q[k][i] = static_cast<T>(s * Q[k][j] + c * Q[k][i]);
            Q[k][j] = temp;
        }
    }
}
```

Можно сохранить параметры вращения параметры вращения $\cos\theta$ и $\sin\theta$ и позже, после накопления матрицы R, восстановить матрицы вращения и вычислить матрицу Q.

Для этого не потребуется лишней памяти, параметры вращения $\cos\theta$ и $\sin\theta$ можно сохранить в позиции R_{ij} , брать в цикле для восстановления матриц вращения и вычисления Q, занулять элемент R_{ij} . Для этого $\cos\theta$ и $\sin\theta$ по формуле превратим в одно число:

$$\tau = \frac{s}{1+c}.$$

Обратная его трансформация в $\cos\theta$ и $\sin\theta$ выполняется по формулам:

$$c = \frac{1-\tau^2}{1+\tau^2},$$

$$s = \frac{2 \cdot \tau}{1 + \tau^2}$$

В результате код будет выглядеть следующим образом:

```
for (int j = 0; j < N - 1; ++j) {
```

```

for (int i = j + 1; i < N; ++i) {
    // приведение к double для точных проверок и вычислений
    double Rjj = static_cast<double>(R[j][j]);
    double Rij = static_cast<double>(R[i][j]);

    // проверки малости (пороги для double)
    if (std::abs(Rij) < 1e-11)
        continue;

    double sqrt_val = std::sqrt(Rjj * Rjj + Rij * Rij);
    if (std::abs(sqrt_val) < 1e-11)
        continue;

    double c = Rjj / sqrt_val;
    double s = -Rij / sqrt_val;

#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
    for (int k = j; k < N; ++k) {
        T temp = static_cast<T>(R[j][k] * c - R[i][k] * s);
        R[i][k] = static_cast<T>(R[j][k] * s + R[i][k] * c);
        R[j][k] = temp;
    }

    double tau = s / (1.0 + c);
    R[i][j] = static_cast<T>(tau);
}

for (int j = 0; j < N - 1; ++j)
{
    for (int i = j + 1; i < N; ++i)
    {
        if (std::abs(R[i][j]) < 1e-11) continue;

        double tau = static_cast<double>(R[i][j]);
        double tau2 = tau * tau;
        double denom = 1.0 + tau2;
        double c = (1.0 - tau2) / denom;
        double s = 2.0 * tau / denom;

#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int k = 0; k < N; ++k)
        {
            T temp = static_cast<T>(c * Q[k][j] - s * Q[k][i]);
            Q[k][i] = static_cast<T>(s * Q[k][j] + c * Q[k][i]);
            Q[k][j] = temp;
        }
        R[i][j] = T(0);
    }
}

```

Кроме того, хранение в памяти матрицы Q по столбцам с точки зрения устройства компьютерной памяти совсем не удачно — элементы разделены, не будет оптимизации по попаданию нескольких подряд лежащих элементов в кэш. Если хранить матрицу Q транспонированной, обращение будет к элементам одной строки, которые в памяти будут лежать рядом. Должно смениться обращение:

$$Q[k][i] \rightarrow Q[i][k],$$

$$Q[k][j] \rightarrow Q[j][k].$$

А также в конце выполнения разложения необходимо сделать транспонирование:

```
for (int i = 0; i < N; ++i)
{
    for (int j = 0; j < i; ++j)
        std::swap(Q[i][j], Q[j][i]);
}
```

Эксперимент показал, что наибольший выигрыш по времени для размеров диагонали N матрицы A больше 1000 элементов даёт способ, согласно которому нужно вычислять матрицу Q отдельно от матрицы R и хранить Q транспонированной.

Таблица с замерами времени на восьми потоках в зависимости от размера данных (матрица NxN чисел с плавающей точкой двойной точности) представлена ниже. Также для сравнения прикреплён результат по времени предыдущей оптимизированной реализации QR-разложения с использованием метода отражений Хаусхолдера:

Размер данных (матрица NxN)	Время работы, с	Абсолютная ошибка	Относительная ошибка	Время работы QR-разложения на основе отражений Хаусхолдера, с
100	0.0022843	8.58151e-13	1.47692e-15	0.0013707
200	0.0144698	2.35456e-12	2.03474e-15	0.0078925
300	0.0419104	4.5015e-12	2.58997e-15	0.0239516
400	0.0981446	6.80949e-12	2.93754e-15	0.0492212
500	0.166818	9.5604e-12	3.29942e-15	0.0921322
1000	1.44459	2.69627e-11	4.65961e-15	0.189128
1500	3.84659	5.03343e-11	5.79791e-15	0.776455
2000	8.2133	7.51954e-11	7.28252e-15	2.38406
2500	14.1009	1.05371e-10	1.49371e-15	4.97292
3000	21.5005	1.40569e-10	8.09457e-15	9.53905
3500	30.9923	1.79351e-10	8.85388e-15	16.6415
4000	42.9903	2.18755e-10	9.44817e-15	24.5809

Таблица 1. Замеры для базовой версии метода Гивенса

Видно, что даже на небольших данных эта версия проигрывает оптимизированному методу Хаусхолдера, а на больших данных замедление более чем в два раза. В следующих реализациях будет производиться оптимизация узкого места несложного алгоритма QR-

разложения на основе метода вращений Гивенса — обращения к памяти во время вращения строк матриц Q и R .

Реализация прикреплена в Приложении 1.

Хранение Q и R в одной структуре

Для уменьшения времени работы программы можно попробовать использовать структуру данных, которая обеспечит лучшее обращение к памяти. Алгоритм использует специализированную структуру данных QRMatrix, которая объединяет хранение матриц Q и R для оптимизации доступа к памяти:

```
template <class T>
class QRMatrix
{
public:
    QRMatrix(const std::vector<std::vector<T>>& A)
        : _data(A.size()* A.size())
        , N(A.size())
    {
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int i = 0; i < N; ++i)
        {
            QAt(i, i) = 1.0;
            for (int j = 0; j < N; ++j)
            {
                RAt(i, j) = A[i][j];
            }
        }

        T& RAt(size_t row, size_t col)
        {
            return _data[row * N + col].r;
        }
        T& QAt(size_t row, size_t col) {
            return _data[col * N + row].q;
        }

private:
    struct QRElement {
        T q;
        T r;
    };
    std::vector<QRElement> _data;
    size_t N;
};
```

При этом матрица R хранится по строкам, а матрица Q, транспонированная, по столбцам.

Применение вращения затрагивает только две строки матрицы R и два столбца матрицы Q. Для матрицы R обновляются элементы с индексами $k \geq j$, тогда как для матрицы Q преобразуются все элементы соответствующих столбцов. Реализация включает несколько уровней проверок численной устойчивости: игнорируются вращения для пренебрежимо

малых элементов и контролируется относительная величина элементов для предотвращения потери точности.

Небольшой выигрыш по ускорению может дать совместное попарное хранение элементов Q и R благодаря оптимизации доступа к памяти. При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи. Это и совместное обновление матриц в одном цикле может дать хорошее ускорение для этой реализации.

Однако данный метод не дал желаемый результат. Замеры времени также на восьми потоках (матрица $N \times N$ чисел с плавающей точкой двойной точности) приведены в таблице ниже. Также для сравнения прикреплён результат по времени предыдущей реализации методом вращений Гивенса и оптимизированного QR-разложения на основе отражений Хаусхолдера в тех же условиях (восемь потоков, числа с плавающей точкой двойной точности):

Размер данных (матрица $N \times N$)	Время работы, с	Абсолютная ошибка	Относительная ошибка	Время работы первой реализации на основе вращений Гивенса, с	Время работы QR-разложения на основе отражений Хаусхолдера, с
100	0.0013685	8.56541e-13	1.47692e-15	0.0022843	0.0013707
200	0.0096698	2.4303e-12	2.1026e-15	0.0144698	0.0078925
300	0.0418232	4.34459e-12	2.50572e-15	0.0419104	0.0239516
400	0.0906898	6.69974e-12	2.89391e-15	0.0981446	0.0492212
500	0.171549	9.33753e-12	3.22679e-15	0.166818	0.0921322
1000	1.02815	2.71479e-11	4.6936e-15	1.44459	0.189128
1500	2.76514	4.98842e-11	5.74576e-15	3.84659	0.776455
2000	5.62658	7.57192e-11	6.54098e-15	8.2133	2.38406
2500	10.2853	1.06426e-10	7.35348e-15	14.1009	4.97292
3000	17.7229	1.4084e-10	8.10977e-15	21.5005	9.53905
3500	25.7584	1.79638e-10	8.86769e-15	30.9923	16.6415
4000	36.8808	2.19198e-10	9.46739e-15	42.9903	24.5809

Таблица 2. Замеры для версии метода Гивенса с улучшенным обращением к памяти матриц Q и R

Теория подтвердилась, улучшение локальности таким способом (объединённое хранение матриц Q и R) немного ускорило работу программы по сравнению с первой реализацией QR-разложения на основе вращений Гивенса. Заметнее всего это ускорение на данных более 2000×2000 элементов. По-прежнему заметно, что алгоритм на основе вращений значительно уступает алгоритму на основе отражений Хаусхолдера.

Реализация прикреплена в Приложении 2.

Использование SIMD

Для векторных вычислений можно использовать встроенную в OpenMP векторизацию или использовать AVX-интринсики.

Использование `#pragma omp simd` перед циклом даст явно знать компилятору, что этот цикл может быть векторизован. Директива указывает, что итерации цикла независимы и могут быть выполнены параллельно с использованием векторных инструкций процессора. Циклы вращения строк матриц Q и R из базовой версии могут без изменения быть использованы в SIMD-версии:

```
// вычисление матрицы R путём вращения строк
#pragma omp simd
for (int k = j; k < N; ++k)
{
    T temp = static_cast<T>(R[j][k] * c - R[i][k] * s);
    R[i][k] = static_cast<T>(R[j][k] * s + R[i][k] * c);
    R[j][k] = temp;
}

// вычисление матрицы Q_T путём вращения строк
#pragma omp simd
for (int k = 0; k < N; ++k)
{
    T temp = static_cast<T>(c * Q_T[j][k] - s * Q_T[i][k]);
    Q_T[i][k] = static_cast<T>(s * Q_T[j][k] + c * Q_T[i][k]);
    Q_T[j][k] = temp;
}
```

Для `#pragma openmp simd` есть клауза `aligned`, помечающая, что память, над которой производятся вычисления, выровнена по границе размера типа (64-кратные адреса для 64-битных чисел с плавающей точкой двойной точности и 32-кратные адреса для 32-битных чисел с плавающей точкой одинарной точности). Для ещё более лучшего обращения к памяти следует выравнивать строки матриц Q и R по границам 64 или 32, в зависимости от типа. Например, для 8-байтного double:

```
// выделение памяти с выравниванием строк
constexpr size_t elem_size = sizeof(T);
constexpr size_t align_mod = 64 / elem_size;

size_t stride = N; // каждая строка будет начинаться с выровненного адреса
size_t rem = N % align_mod;
if (rem != 0) stride += align_mod - rem; // stride кратен 8

// обёртки для автоматического освобождения памяти
auto deleter = [](T* p) { ::operator delete[](p, c_align); };
std::unique_ptr<T[], decltype(deleter)> R_data{
    static_cast<T*> (::operator new[](N * stride * elem_size, c_align)), deleter };
std::unique_ptr<T[], decltype(deleter)> Q_data{
    static_cast<T*> (::operator new[](N * stride * elem_size, c_align)), deleter };

#define R(i, j) R_data[(i)*stride + (j)] // функция доступа: R_data[i * stride + j]
#define Q_T(i, j) Q_data[(i)*stride + (j)] // Q в транспонированном виде
```

Тогда вращение строк при выровненной памяти и использовании simd из OpenMP одновременно для матриц Q и R в одном цикле будет выглядеть следующим образом:

```
// локальные указатели на строки с выравниванием
T* __restrict Rj_ptr = &R_data[j * stride];
T* __restrict Ri_ptr = &R_data[i * stride];
T* __restrict Qj_ptr = &Q_data[j * stride];
T* __restrict Qi_ptr = &Q_data[i * stride];

#pragma omp simd aligned(Rj_ptr, Ri_ptr, Qj_ptr, Qi_ptr : 64)
for (size_t k = 0; k < stride; ++k)
{
    const T Rj = Rj_ptr[k];
    const T Ri = Ri_ptr[k];
    const T Qj = Qj_ptr[k];
    const T Qi = Qi_ptr[k];

    Rj_ptr[k] = Rj * c - Ri * s;
    Ri_ptr[k] = Rj * s + Ri * c;
    Qj_ptr[k] = Qj * c - Qi * s;
    Qi_ptr[k] = Qj * s + Qi * c;
}
```

Векторную обработку также можно добавить с помощью интринсиков, используя специальные функции, которые самостоятельно производят работу с памятью и регистрами. Идея их использования та же: при вращении строк матриц Q и R, производя одновременно вычисления четырёх (при работе с вещественными числами двойной точности) или восьми (при работе с вещественными числами одинарной точности) значений в двух строках одновременно:

```
// data type is double
__m256d Rj = _mm256_loadu_pd(&Rj_ptr[k]);
__m256d Ri = _mm256_loadu_pd(&Ri_ptr[k]);

__m256d c_vec = _mm256_set1_pd(c);
__m256d s_vec = _mm256_set1_pd(s);

__m256d new_Rj = _mm256_fmsub_pd(Rj, c_vec, _mm256_mul_pd(Ri, s_vec));
__m256d new_Ri = _mm256_fmadd_pd(Rj, s_vec, _mm256_mul_pd(Ri, c_vec));

_mm256_storeu_pd(&Rj_ptr[k], new_Rj);
_mm256_storeu_pd(&Ri_ptr[k], new_Ri);

// data type is float
__m256 Rj = _mm256_loadu_ps(&Rj_ptr[k]);
__m256 Ri = _mm256_loadu_ps(&Ri_ptr[k]);

__m256 c_vec = _mm256_set1_ps(c);
__m256 s_vec = _mm256_set1_ps(s);

__m256 new_Rj = _mm256_fmsub_ps(Rj, c_vec, _mm256_mul_ps(Ri, s_vec));
__m256 new_Ri = _mm256_fmadd_ps(Rj, s_vec, _mm256_mul_ps(Ri, c_vec));

_mm256_storeu_ps(&Rj_ptr[k], new_Rj);
_mm256_storeu_ps(&Ri_ptr[k], new_Ri);
```

`__m256d` (256 бит для хранения 8-байтных вещественных чисел двойной точности) и `__m256` (256 бит для хранения 4-байтных вещественных чисел одинарной точности) — это типы данных, позволяющие работать с AVX-интринсиками. Функции с префиксом `_mm256_*` позволяют загружать и читать данные из регистра, производить над ними математические операции сложения, умножения и др.

Эта операция выполняется одновременно для столько чисел, сколько помещается в 256-битном регистре, оставшийся «хвост» обрабатывается старым способом, без векторизации:

```
// data type is double
if constexpr (std::is_same_v<T, double>)
{
    size_t k = j;
    for (; k + 3 < N; k += 4) // 4 values
    {
        // data processing with AVX-registers
    }

    for (; k < N; ++k) // values that remains
    {
        // data processing with scalar operations
    }
}
else if constexpr (std::is_same_v<T, float>)
{
    size_t k = j;
    for (; k + 7 < N; k += 8) // 8 values
    {
        // data processing with AVX-registers
    }

    for (; k < N; ++k)
    {
        // data processing with scalar operations
    }
}
```

Точно тот же подход выполняется для восстановления с помощью матриц вращения матрицы Q:

```

// применяем вращение строк Q_T
if constexpr (std::is_same_v<T, double>)
{
    size_t k = 0;
    for (; k + 3 < N; k += 4)
    {
        __m256d Qj = _mm256_loadu_pd(&Qj_ptr[k]);
        __m256d Qi = _mm256_loadu_pd(&Qi_ptr[k]);

        __m256d c_vec = _mm256_set1_pd(c);
        __m256d s_vec = _mm256_set1_pd(s);

        __m256d new_Qj = _mm256_fmsub_pd(Qj, c_vec, _mm256_mul_pd(Qi, s_vec));
        __m256d new_Qi = _mm256_fmadd_pd(Qj, s_vec, _mm256_mul_pd(Qi, c_vec));

        _mm256_storeu_pd(&Qj_ptr[k], new_Qj);
        _mm256_storeu_pd(&Qi_ptr[k], new_Qi);
    }
    for (; k < N; ++k)
    {
        T Qj = Qj_ptr[k], Qi = Qi_ptr[k];
        Qj_ptr[k] = Qj * c - Qi * s;
        Qi_ptr[k] = Qj * s + Qi * c;
    }
}
else if constexpr (std::is_same_v<T, float>)
{
    size_t k = 0;
    for (; k + 7 < N; k += 8)
    {
        __m256 Qj = _mm256_loadu_ps(&Qj_ptr[k]);
        __m256 Qi = _mm256_loadu_ps(&Qi_ptr[k]);

        __m256 c_vec = _mm256_set1_ps(c);
        __m256 s_vec = _mm256_set1_ps(s);

        __m256 new_Qj = _mm256_fmsub_ps(Qj, c_vec, _mm256_mul_ps(Qi, s_vec));
        __m256 new_Qi = _mm256_fmadd_ps(Qj, s_vec, _mm256_mul_ps(Qi, c_vec));

        _mm256_storeu_ps(&Qj_ptr[k], new_Qj);
        _mm256_storeu_ps(&Qi_ptr[k], new_Qi);
    }
    for (; k < N; ++k)
    {
        T Qj = Qj_ptr[k], Qi = Qi_ptr[k];
        Qj_ptr[k] = Qj * c - Qi * s;
        Qi_ptr[k] = Qj * s + Qi * c;
    }
}

```

К сожалению, мне не удалось заставить мой компилятор использовать `#pragma omp simd` с клаузой `aligned`, при компиляции появлялось предупреждение о том, что она игнорируется.

Тем не менее, использование `simd` от OpenMP показало лучший результат по сравнению с предыдущими реализациями QR разложения на основе вращений Гивенса. Однако незначительный разрыв по времени есть, между использованием интринсиков и `simd` прагмы есть небольшая разница. Лучшее время всё-таки показала реализация с интринсиками.

Замеры времени также на восьми потоках (матрица $N \times N$ чисел с плавающей точкой двойной точности) находятся в таблице ниже. Также для сравнения прикреплён результат по времени предыдущей реализации методом вращений Гивенса и оптимизированного QR-разложения на основе отражений Хаусхолдера в тех же условиях (восемь потоков, числа с плавающей точкой двойной точности):

Размер данных (матрица $N \times N$)	Время работы, с	Абсолютная ошибка	Относительная ошибка	Время работы QR-разложения с вращениями Гивенса с улучшенным обращением к памяти, с	Время работы QR-разложения на основе отражений Хаусхолдера, с
100	0.0007243	8.80689e-13	1.5122e-15	0.0013685	0.0009344
200	0.0031774	2.4283e-12	2.10109e-15	0.0096698	0.0088197
300	0.013791	4.49498e-12	2.58821e-15	0.0418232	0.0287179
400	0.0374398	6.93339e-12	2.99066e-15	0.0906898	0.073563
500	0.0994119	9.22069e-12	3.18535e-15	0.171549	0.139082
1000	0.537822	2.70847e-11	4.67937e-15	1.02815	0.171962
1500	1.72604	4.99482e-11	5.75594e-15	2.76514	0.667425
2000	4.33583	7.54317e-11	6.51629e-15	5.62658	2.80849
2500	8.67453	1.06775e-10	7.37974e-15	10.2853	5.58732
3000	15.1124	1.41068e-10	8.12511e-15	17.7229	11.3973
3500	24.238	1.78101e-10	8.79166e-15	25.7584	18.1994
4000	36.0047	2.19433e-10	9.47739e-15	36.8808	27.4943

Таблица 3. Замеры для версии метода Гивенса с векторизованной памятью

Видно, что новой оптимизации удалось отработать за лучшее меньшее время, по сравнению со второй реализацией QR-разложения на основе вращений Гивенса, с улучшенным обращением к памяти. Для небольших данных (матриц 500×500 и меньше) этот алгоритм превосходит даже оптимизированный алгоритм на основе отражений Хаусхолдера. Однако на данных больше 1000×1000 элементов всё ещё лучшим показывает себя QR-разложение на основе отражений Хаусхолдера.

Реализация прикреплена в Приложении 3.

Сводная таблица ниже содержит сравнение всех полученных реализаций QR-разложения на основе вращений Гивенса по времени в секундах, а также время работы QR-разложения на основе отражений Хаусхолдера и QR-разложения из библиотеки SciLab.

Размер данных (матрица N×N)	Время работы первой реализации QR- разложения с вращениями Гивенса , с	Время работы QR- разложения с вращениями Гивенса с улучшенным обращением к памяти, с	Время работы QR- разложения с вращениями Гивенса с векторизацие й, с	Время работы QR- разложения на основе отражений Хаусхолдера, с	Время работы QR- разложения из библиотеки SciLab, с
100	0.0022843	0.0013685	0.0007243	0.0009344	0.0335941
200	0.0144698	0.0096698	0.0031774	0.0088197	0.0039998
300	0.0419104	0.0418232	0.013791	0.0287179	0.0084994
400	0.0981446	0.0906898	0.0374398	0.073563	0.0054738
500	0.166818	0.171549	0.0994119	0.139082	0.0088187
1000	1.44459	1.02815	0.537822	0.171962	0.0412016
1500	3.84659	2.76514	1.72604	0.667425	0.0998157
2000	8.2133	5.62658	4.33583	2.80849	0.1859652
2500	14.1009	10.2853	8.67453	5.58732	0.3497936
3000	21.5005	17.7229	15.1124	11.3973	0.6075179
3500	30.9923	25.7584	24.238	18.1994	0.9034108
4000	42.9903	36.8808	36.0047	27.4943	1.4172756

Таблица 4. Сравнение версий метода Гивенса с версиями метода Хаусхолдера и из библиотеки SciLab

Видно, что самый оптимальный и устойчивый результат показывает метод из библиотеки SciLab. В SciLab QR-разложение реализуется с использованием подпрограмм из

библиотеки Lapack. Для вещественных матриц используются процедуры DGEQRF, DGEQPF и DORGQR, а для комплексных — ZGEQRF, ZGEQPF и ZORGQR.

Некоторые особенности реализации в Scilab:

- Если матрица X имеет размер $m \times n$ и $m > n$, то вычисляются только первые n столбцов матрицы Q и первые n строк матрицы R (так называемый «экономичный» режим).
- Если столбец матрицы X является линейной комбинацией предыдущих столбцов, то элементы матрицы R после определённого столбца будут равны нулю, и в этом случае R будет верхней трапецевидной.
- Если матрица X имеет ранг rk , то строки $R(rk+1, :)$, $R(rk+2, :)$ и т. д. будут нулевыми.

QR-разложение Хессенберговой матрицы

Верхняя матрица Хессенберга — это квадратная матрица $N \times N$, у которой $a_{ij} = 0$, если $i > j + 1$.

Нижняя матрица Хессенберга — это квадратная матрица $N \times N$, у которой $a_{ij} = 0$, если $j > i + 1$.

Матрица, являющаяся одновременно и верхней, и нижней матрицами Хессенберга, трёхдиагональна.

Код функции для генерации верхних и/или нижних матриц хессенберга представлен ниже:

```
template <typename T>
std::vector<std::vector<T>> generateHassenbergMatrix(size_t size, bool upper, bool lower)
{
    std::srand(std::time(0));
    std::vector<std::vector<T>> res(size, std::vector<T>(size, 0));
    size_t N = size, M = size;
    if (upper == true && lower == true)
    {
        for (size_t i = 0; i < N; ++i)
        {
            if (i != 0) res[i][i - 1] = randomNotZeroValue<T>();
            if (i != N - 1) res[i][i + 1] = randomNotZeroValue<T>();
            res[i][i] = randomNotZeroValue<T>();
        }
    }
    else if (upper == true)
    {
        for (size_t i = 0; i < N; ++i)
        {
            if (i != 0) res[i][i - 1] = randomNotZeroValue<T>();
            for (size_t j = i; j < M; ++j)
            {
                res[i][j] = randomNotZeroValue<T>();
            }
        }
    }
    else if (lower == true)
    {
        for (size_t i = 0; i < N; ++i)
        {
            if (i != N - 1) res[i][i + 1] = randomNotZeroValue<T>();
            for (size_t j = 0; j < i + 1; ++j)
            {
                res[i][j] = randomNotZeroValue<T>();
            }
        }
    }
    return res;
}
```

Сводная таблица ниже содержит сравнение времени решения задачи разложения верхней хессенберговой матрицы на Q и R для самых оптимальных версий QR-разложения на основе вращений Гивенса и отражений Хаусхолдера, а также версии из библиотеки SciLab.

Размер данных (матрица NxN)	Время работы QR- разложения с вращениями Гивенса с векторизацией, с	Время работы QR- разложения на основе отражений Хаусхолдера, с	Время работы QR- разложения из библиотеки SciLab, с
100	0.0001294	0.0009429	0.030026
200	0.0004733	0.0076185	0.003412
300	0.0012873	0.0222422	0.0045437
400	0.0014392	0.0370319	0.0069478
500	0.0022636	0.0707061	0.0107381
1000	0.0099787	0.193794	0.0422846
1500	0.0244809	1.14657	0.07312
2000	0.049296	6.44008	0.1638567
2500	0.070357	16.1027	0.3190695
3000	0.111763	28.8915	0.533518
3500	0.165066	42.731	0.7996886
4000	0.226181	67.3722	1.7190395

Таблица 5. Сравнение скорости решения QR-разложения нижней хессенберговой матрицы методом Гивенса, методом Хаусхолдера и QR разложением из библиотеки SciLab

Видно, что моя реализация QR-разложения методом вращений Гивенса решила задачу QR-разложения верхней хессенберговой матрицы быстрее, так как, по-видимому, SciLab использует метод отражений Хаусхолдера, который не может игнорировать отдельно стоящие пустые значения в столбцах, а вынужден производить отражение и занулять весь столбец целиком, даже не смотря на то, что в нём всего один значащий элемент.

Сводная таблица ниже содержит сравнение времени решения задачи разложения нижней хессенберговой матрицы на Q и R для самых оптимальных версий QR-разложения на основе вращений Гивенса и отражений Хаусхолдера, а также версии из библиотеки SciLab.

Размер данных (матрица NxN)	Время работы QR- разложения с вращениями Гивенса с векторизацией, с	Время работы QR- разложения на основе отражений Хаусхолдера, с	Время работы QR- разложения из библиотеки SciLab, с
100	0.0003957	0.0013103	0.0179448
200	0.0023856	0.0069098	0.0065019
300	0.0092742	0.018879	0.0073951
400	0.0168637	0.0363722	0.0080666
500	0.0427091	0.0720739	0.0111006
1000	0.510321	0.173632	0.0324745
1500	1.63854	0.643838	0.0849378
2000	3.54986	2.10122	0.1808189
2500	7.19002	5.00486	0.3221421
3000	12.8882	9.74322	0.4588081
3500	20.7162	16.1714	0.7511698
4000	31.2464	24.6837	1.093605

Таблица 6. Сравнение скорости решения QR-разложения нижней хессенберговой матрицы методом Гивенса, методом Хаусхолдера и QR разложением из библиотеки SciLab

Видно, что моя реализация QR-разложения методом вращений Гивенса справилась с решением задачи QR-разложения нижней хессенберговой матрицы хуже, опять по той же причине, что SciLab использует метод отражений Хаусхолдера, который быстрее зануляет целые столбцы.

Заключение

В заключение хотелось бы подытожить полученные результаты.

Различные версии алгоритма QR-разложения на основе вращений Гивенса в целом работают быстрее чем, QR-разложение на основе отражений Хаусхолдера, на небольших данных ($< 1000 \times 1000$ элементов), однако всё ещё проигрывает по скорости на больших ($>> 1000 \times 1000$ элементов).

Дело в ограниченной применимости метода. В виду его особенности обнуления отдельных элементов вместо целых столбцов под диагональю, как в методе Хаусхолдера, этот метод даст лучший результат при применении его к разреженным матрицам. Игнорирование нулевых отдельно стоящих значений позволит выполнить QR разложение быстрее. Так работают, например, открытые известные реализации: при работе с плотными матрицами используется метод Хаусхолдера, а при работе с разреженными матрицами — метод Гивенса.

Также можно попробовать реализовать блочный алгоритм для дальнейшего нивелирования эффекта от обращений к памяти во время выполнения вращений Гивенса. Потребуется использовать согласованную общую память, разделяемую отдельными потоками, отвечающими за разные блоки. Для этого больше подойдёт MPI-параллелизм на уровне процессов.

Полный код работы можно посмотреть по ссылке <https://github.com/wellcome2AD/QR-decomposition>.

Список литературы

1. Horn, R. A., Johnson C. R. Matrix analysis: Textbook / Roger A. Horn, Charles R. Johnson. – 2nd ed. — Cambridge University Press, 2012. — 561 p. — URL: <https://doi.org/10.1017/CBO9780511810817> (дата обращения: 14.12.2024).
2. Белов, С.А., Золотых Н.Ю. Численные методы линейное алгебры. Лабораторный практикум. / С.А. Белов, Н.Ю. Золотых — Нижний Новгород: Изд-во Нижегородского госуниверситета им. Н.И. Лобачевского, 2005. — 264 с.
3. Agullo, E. Exploiting a Parametrized Task Graph model for the parallelization of a sparse direct multifrontal solver // Euro-Par 2016: Parallel Processing Workshops, 2017. – P. 175–186. – URL: https://doi.org/10.1007/978-3-319-58943-5_14 (дата обращения: 14.12.2024).
4. Davis, T. A. Algorithm 915, SuiteSparseQR: Multifrontal multithreaded rank-revealing sparse QR factorization / T. A. Davis // ACM Transactions on Mathematical Software, Vol. 38, No. 1, Article 8, 2011. – P. 1-22. – URL: <https://doi.org/10.1145/2049662.2049670> (дата о б р а щ е н и я : 1 4 . 1 2 . 2 0 2 4) .
5. Новак, А. Е. Параллельный алгоритм построения QR-разложения разреженных матриц для систем с общей памятью / А. Е. Новак, С. А. Лебедев // Суперкомпьютерные дни в России : Труды международной конференции, Москва, 24–25 сентября 2018 года / Суперкомпьютерный консорциум университетов России, Российская академия наук. – Москва: Московский государственный университет имени М.В. Ломоносова" Издательский Дом (типография), 2018. – С. 841-847. – URL: <https://2018.russianscdays.org/files/pdf18/841.pdf> (дата обращения: 14.12.2025).
6. Soliverez, C. E. Orthonormalization on the plane: a geometric approach / C. E. Soliverez, E. Gagliano // Revista Mexicana de Física., 1985. – P. 743–758. – URL: https://www.researchgate.net/publication/267471614_Orthonormalization_on_the_plane_a_geometric_approach (д а т а о б р а щ е н и я : 1 4 . 1 2 . 2 0 2 4) .
7. Вержбицкий, В. М. Численные методы. Линейная алгебра и нелинейные уравнения : Учебник / В. М. Вержбицкий. – Москва : Высшая школа, 2005. – 432 с.
8. Фролов, А. В. Сопоставление разных методов решения одной задачи по методике проекта Algowiki / А. В. Фролов, А. С. Антонов, Вл. В. Воеводин, А. М. Теплов // Параллельные вычислительные технологии (ПаВТ'2016), 2016 – С. 347–360. – URL: <https://ceur-ws.org/Vol-1576/062.pdf> (дата обращения: 14.12.2024).

Приложение 1.

Базовая параллельная реализация QR-разложения методом вращений Гивенса.

```
#pragma once

#include <vector>
#include <cmath>

#include "../QRSolver.h"
#include "math.h"

template <typename T>
class GivensMethodBasic : public QRSolver<T> {
public:
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
        std::vector<std::vector<T>>& Q,
        std::vector<std::vector<T>>& R) override
    {
        auto N = A.size();
        // матрица Q будет храниться транспонированной
        Q = std::vector<std::vector<T>>(N, std::vector<T>(N, 0));
        R = A;

        for (int i = 0; i < N; ++i)
            Q[i][i] = T(1);

        for (int j = 0; j < N - 1; ++j) {
            for (int i = j + 1; i < N; ++i) {
                // приведение к double для точных проверок и вычислений
                double Rjj = static_cast<double>(R[j][j]);
                double Rij = static_cast<double>(R[i][j]);

                // проверки малости (пороги для double)
                if (std::abs(Rij) < 1e-11)
                    continue;

                double sqrt_val = std::sqrt(Rjj * Rjj + Rij * Rij);
                if (std::abs(sqrt_val) < 1e-11)
                    continue;

                double c = Rjj / sqrt_val;
                double s = -Rij / sqrt_val;

                // вычисление матрицы R путём вращения строк
                #pragma omp parallel for num_threads(thread_num) if (N >= 1000)
                for (int k = j; k < N; ++k) {
                    T temp = static_cast<T>(R[j][k] * c - R[i][k] * s);
                    R[i][k] = static_cast<T>(R[j][k] * s + R[i][k] * c);
                    R[j][k] = temp;
                }

                //std::cout << "expected R, rotation (" << i << ", " << j << "):";
                //printMatrix(R);
                //std::cout << std::endl;

                // сохранение коэффициентов вращения
                double tau = s / (1.0 + c);
                R[i][j] = static_cast<T>(tau);
            }
        }
    }
}
```



```

for (int j = 0; j < N - 1; ++j)
{
    for (int i = j + 1; i < N; ++i)
    {
        if (std::abs(R[i][j]) < 1e-11) continue;
        // восстановление коэффициентов вращения
        double tau = static_cast<double>(R[i][j]);
        double tau2 = tau * tau;
        double denom = 1.0 + tau2;
        double c = (1.0 - tau2) / denom;
        double s = 2.0 * tau / denom;

        // вычисление матрицы Q_T путём вращения строк
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int k = 0; k < N; ++k)
        {
            // обращение к элементам транспонированной матрицы Q
            T temp = static_cast<T>(c * Q[j][k] - s * Q[i][k]);
            Q[i][k] = static_cast<T>(s * Q[j][k] + c * Q[i][k]);
            Q[j][k] = temp;
        }
        R[i][j] = T(0);
    }
}

// транспонирование
for (int i = 0; i < N; ++i)
{
    for (int j = 0; j < i; ++j)
        std::swap(Q[i][j], Q[j][i]);
}

};

```

Приложение 2.

Версия QR-разложения с помощью метода Гивенса, где элементы Q и R хранятся попарно рядом в памяти.

```
#pragma once

#include <vector>
#include <cmath>

#include "../IQRSolver.h"
#include "math.h"

template <class T>
class QRMatrix
{
public:
    QRMatrix(const std::vector<std::vector<T>>& A)
        : _data(A.size()* A.size())
        , N(A.size())
    {
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int i = 0; i < N; ++i)
        {
            QAt(i, i) = 1.0;
            for (int j = 0; j < N; ++j)
            {
                RAt(i, j) = A[i][j];
            }
        }

        T& RAt(size_t row, size_t col)
        {
            return _data[row * N + col].r;
        }
        T& QAt(size_t row, size_t col) {
            return _data[col * N + row].q;
        }

private:
    struct QRElement {
        T q;
        T r;
    };
    std::vector<QRElement> _data;
    size_t N;
};

template <typename T>
class GivensMethodQRInOneMatrix : public IQRSolver<T> {
public:
    // оптимизированная версия. уменьшены обращения к памяти. QR хранятся в одной структуре, каждый
    // элемент попарно рядом в памяти. Q записана по столбцам, R -- по строкам.
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A, std::vector<std::vector<T>>& Q,
std::vector<std::vector<T>>& R) override
    {
        auto N = A.size();
        QRMatrix<T> qr(A);
```

```

for (auto j = 0; j < N - 1; j++)
{ // зануляем весь столбец j
    for (auto i = j + 1; i < N; i++)
    { // под главной диагональю
        double Rjj = qr.RAt(j, j);
        double Rij = qr.RAt(i, j);

        if (std::abs(Rij) < 1e-11 * std::max(std::abs(Rjj), 1.0)) {
            continue; // относительные числа
        }

        if (std::abs(Rij) < 1e-11 * std::abs(Rjj)) { // R[i][j] пренебрежимо мал по сравнению с
R[j][j]
            continue;
        }

        if (std::abs(Rij) < 1e-11) { // уже 0
            continue;
        }

        auto sqrt = std::sqrt(Rjj * Rjj + Rij * Rij);
        if (std::abs(sqrt) < 1e-11) { // для корректировки ошибки игнорируется малый
знаменатель
            continue;
        }
        double c = Rjj / sqrt, s = -Rij / sqrt;

#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (auto k = 0; k < N; k++)
        {
            if (k >= j)
            { // меняются только две строки -- i и j
                auto temp = qr.RAt(j, k) * c - qr.RAt(i, k) * s;
                qr.RAt(i, k) = qr.RAt(j, k) * s + qr.RAt(i, k) * c;
                qr.RAt(j, k) = temp;
            }

            // меняются только два столбца -- i и j
            auto temp = c * qr.QAt(k, j) - s * qr.QAt(k, i);
            qr.QAt(k, i) = s * qr.QAt(k, j) + c * qr.QAt(k, i);
            qr.QAt(k, j) = temp;
        }
    }
}

R = Q = std::vector<std::vector<T>>>(N, std::vector<T>(N));
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
for (int i = 0; i < N; ++i)
{
    for (int j = 0; j < N; ++j)
    {
        R[i][j] = qr.RAt(i, j);
        Q[i][j] = qr.QAt(i, j);
    }
}
};

```

Приложение 3.

Версия QR-разложения с помощью метода Гивенса, в которой используется подход Single-Instruction-Multiple-Data с помощью интринсиков.

```
#pragma once

#include <vector>
#include <cmath>
#include <memory>
#include <immintrin.h>
#include <type_traits>

#include "../QRSolver.h"

template <typename T>
class GivensMethodSIMD : public QRSolver<T> {
public:
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
        std::vector<std::vector<T>>& Q,
        std::vector<std::vector<T>>& R) override
    {
        const int N = A.size();
        if (N == 0) return;

        size_t total = static_cast<size_t>(N) * N;
        auto R_data = std::make_unique<T[]>(total);
        auto Q_data = std::make_unique<T[]>(total); // Q_T

        // инициализация: R = A, Q = E
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                R_data[i * N + j] = A[i][j];
                Q_data[i * N + j] = (i == j) ? T(1) : T(0);
            }
        }

        // прямой ход: вычисление R
        for (int j = 0; j < N - 1; ++j) {
            for (int i = j + 1; i < N; ++i) {
                const double Rjj = R_data[j * N + j];
                const double Rij = R_data[i * N + j];

                if (std::abs(Rij) < 1e-11)
                    continue;

                const double sqrt_val = std::sqrt(Rjj * Rjj + Rij * Rij);
                if (std::abs(sqrt_val) < 1e-11)
                    continue;

                const double c = Rjj / sqrt_val;
                const double s = -Rij / sqrt_val;

                T* Rj_ptr = &R_data[j * N];
                T* Ri_ptr = &R_data[i * N];

                // векторизованное вращение строк R (ненулевые значения только в столбцах правее j)
                if constexpr (std::is_same_v<T, double>) {
                    size_t k = j;
                    for (; k + 3 < N; k += 4) {
```

```

__m256d Rj = _mm256_loadu_pd(&Rj_ptr[k]);
__m256d Ri = _mm256_loadu_pd(&Ri_ptr[k]);

__m256d c_vec = _mm256_set1_pd(c);
__m256d s_vec = _mm256_set1_pd(s);

__m256d new_Rj = _mm256_fmsub_pd(Rj, c_vec, _mm256_mul_pd(Ri, s_vec));
__m256d new_Ri = _mm256_fmadd_pd(Rj, s_vec, _mm256_mul_pd(Ri, c_vec));

_mm256_storeu_pd(&Rj_ptr[k], new_Rj);
_mm256_storeu_pd(&Ri_ptr[k], new_Ri);
}
for (; k < N; ++k) {
    T Rj = Rj_ptr[k], Ri = Ri_ptr[k];
    Rj_ptr[k] = Rj * c - Ri * s;
    Ri_ptr[k] = Rj * s + Ri * c;
}
}
else if constexpr (std::is_same_v<T, float>) {
    size_t k = j;
    for (; k + 7 < N; k += 8) {
        __m256 Rj = _mm256_loadu_ps(&Rj_ptr[k]);
        __m256 Ri = _mm256_loadu_ps(&Ri_ptr[k]);

        __m256 c_vec = _mm256_set1_ps(c);
        __m256 s_vec = _mm256_set1_ps(s);

        __m256 new_Rj = _mm256_fmsub_ps(Rj, c_vec, _mm256_mul_ps(Ri, s_vec));
        __m256 new_Ri = _mm256_fmadd_ps(Rj, s_vec, _mm256_mul_ps(Ri, c_vec));

        _mm256_storeu_ps(&Rj_ptr[k], new_Rj);
        _mm256_storeu_ps(&Ri_ptr[k], new_Ri);
    }
    for (; k < N; ++k) {
        T Rj = Rj_ptr[k], Ri = Ri_ptr[k];
        Rj_ptr[k] = Rj * c - Ri * s;
        Ri_ptr[k] = Rj * s + Ri * c;
    }
}
}

// сохраняем c и s в виде tau
double tau = s / (1.0 + c);
R_data[i * N + j] = static_cast<T>(tau);
}
}

// обратный ход: построение Q_T по сохранённым tau
for (int j = 0; j < N - 1; ++j) {
    for (int i = j + 1; i < N; ++i) {
        if (std::abs(R_data[i * N + j]) < 1e-11) continue;

        double tau = static_cast<double>(R_data[i * N + j]);
        double tau2 = tau * tau;
        double denom = 1.0 + tau2;
        double c = (1.0 - tau2) / denom;
        double s = 2.0 * tau / denom; // восстанавливаем c и s

        T* Qj_ptr = &Q_data[j * N];
        T* Qi_ptr = &Q_data[i * N];

        // применяем вращение строк Q_T
        if constexpr (std::is_same_v<T, double>) {
            size_t k = 0;
            for (; k + 3 < N; k += 4) {

```

```

    __m256d Qj = _mm256_loadu_pd(&Qj_ptr[k]);
    __m256d Qi = _mm256_loadu_pd(&Qi_ptr[k]);

    __m256d c_vec = _mm256_set1_pd(c);
    __m256d s_vec = _mm256_set1_pd(s);

    __m256d new_Qj = _mm256_fmsub_pd(Qj, c_vec, _mm256_mul_pd(Qi, s_vec));
    __m256d new_Qi = _mm256_fmadd_pd(Qj, s_vec, _mm256_mul_pd(Qi, c_vec));

    _mm256_storeu_pd(&Qj_ptr[k], new_Qj);
    _mm256_storeu_pd(&Qi_ptr[k], new_Qi);
}
for (; k < N; ++k) {
    T Qj = Qj_ptr[k], Qi = Qi_ptr[k];
    Qj_ptr[k] = Qj * c - Qi * s;
    Qi_ptr[k] = Qj * s + Qi * c;
}
}
else if constexpr (std::is_same_v<T, float>) {
    size_t k = 0;
    for (; k + 7 < N; k += 8) {
        __m256 Qj = _mm256_loadu_ps(&Qj_ptr[k]);
        __m256 Qi = _mm256_loadu_ps(&Qi_ptr[k]);

        __m256 c_vec = _mm256_set1_ps(c);
        __m256 s_vec = _mm256_set1_ps(s);

        __m256 new_Qj = _mm256_fmsub_ps(Qj, c_vec, _mm256_mul_ps(Qi, s_vec));
        __m256 new_Qi = _mm256_fmadd_ps(Qj, s_vec, _mm256_mul_ps(Qi, c_vec));

        _mm256_storeu_ps(&Qj_ptr[k], new_Qj);
        _mm256_storeu_ps(&Qi_ptr[k], new_Qi);
    }
    for (; k < N; ++k) {
        T Qj = Qj_ptr[k], Qi = Qi_ptr[k];
        Qj_ptr[k] = Qj * c - Qi * s;
        Qi_ptr[k] = Qj * s + Qi * c;
    }
}
}

R_data[i * N + j] = T(0); // восстанавливаем чистый ноль
}
}

// копирование в выходные матрицы
Q.assign(N, std::vector<T>(N));
R.assign(N, std::vector<T>(N));
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
for (int i = 0; i < N; ++i) {
    for (int j = 0; j < N; ++j) {
        R[i][j] = R_data[i * N + j];
        Q[i][j] = Q_data[j * N + i]; // транспонирование
    }
}
}
};

```